Python Activity Record: Student Marks Matrix Analysis

Based on your uploaded Student Marks Matrix Analysis.py program, this activity record explains how to store students' marks in a matrix, calculate each student's total and average, identify the top student, and calculate the average marks for each subject.

Student Marks Matrix Analysis .py

1. Problem statement

To develop a Python program that accepts the names and marks of students in Python, Maths, and English. The program calculates each student's total marks and average, identifies the student with the highest total, and calculates the average marks for each subject.

2. Objectives

- To accept student names and subject marks.
- To store marks in a two-dimensional list (matrix).
- To calculate the total and average marks of each student.
- To identify the top student based on total marks.
- To calculate the average marks in each subject.
- To display a student report in a clear format.

3. Assumptions

1. The user enters the number of students as an integer.
2. Every student has marks for three subjects: Python, Maths, and English.
3. Marks are entered as numeric values.
4. All students are evaluated using the same subjects.
5. The top student is the student with the highest total marks.
6. The program assumes valid input; it does not check whether marks fall between 0 and 100.
7. If two students have the same highest total, the program identifies the first one entered.

4. Algorithm

1. Start the program.
2. Create empty lists for student names and marks.
3. Read the number of students.
4. Define the subjects as Python, Maths, and English.
5. For each student:
    - Read the student's name.
    - Read marks for all three subjects.

    - Store the marks in a row.
    - Store the name and row in their respective lists.
6. Create an empty list to store student totals.
7. For each student:
    - Calculate the total marks.
    - Calculate the average marks.
    - Store the total.
    - Display the student's report.
8. If at least one student exists, find and display the top student.
9. For each subject:
    - Add the marks of all students.
    - Calculate the subject average if at least one student exists.
    - Display the subject average.
10. Stop the program.

5. Flowchart

Start

Input number of students

Input each student's name and 3 subject marks

Store marks in a matrix

Calculate each student's total and average

Find the student with the highest total

Calculate the average for each subject

Display the student report and subject averages

Stop

6. Pseudocode

START

Create empty lists: names, marks

INPUT number of students

Set subjects = Python, Maths, English

```
FOR each student:
    INPUT student name
    Create an empty row

    FOR each subject:
        INPUT marks
        Add marks to row
    END FOR

    Add student name to names
    Add row to marks
END FOR

Create an empty list: totals

FOR each student:
    total = sum of student's marks
    average = total / number of subjects
    Add total to totals
    DISPLAY name, marks, total and average
END FOR

IF number of students > 0:
    Find the highest total
    DISPLAY top student's name and total
END IF

FOR each subject:
    Calculate total marks for that subject
    IF number of students > 0:
        Calculate and display subject average
    END IF
```

END FOR

STOP

7. Python source code

```
names = []
marks = []

n = int(input("Number of students: "))
subjects = ["Python", "Maths", "English"]

for i in range(n):
    name = input("Student name: ")
    row = []

    for subject in subjects:
        score = float(input(name + " - " + subject + " marks: "))
        row.append(score)

    names.append(name)
    marks.append(row)

totals = []
print("\n--- STUDENT REPORT ---")

for i in range(n):
    total = sum(marks[i])
    average = total / len(subjects)
    totals.append(total)

    print(names[i], "Marks:", marks[i],
          "Total:", total, "Average:", round(average, 2))
```

```
if n > 0:
    top = totals.index(max(totals))
    print("Top student:", names[top], "with total", totals[top])


for j in range(len(subjects)):
    subject_total = 0


    for i in range(n):
        subject_total += marks[i][j]


    if n > 0:
        print(subjects[j], "average:",
              round(subject_total / n, 2))
```

8. Test cases

Test case	Input marks	Expected result
1	Ravi: 80, 75, 90; Priya: 90, 85, 80	Priya is the top student with 255 marks
2	Arun: 50, 60, 70	Total = 180; average = 60
3	One student with marks 100, 100, 100	Total = 300; average = 100
4	Number of students = 0	No student report or subject averages are displayed

9. Output screenshots

These are sample console output previews, not actual screenshots. Run the program in Python IDLE or VS Code and take screenshots of your own output for your activity record.

Test case 1 — Two students

Sample input: Ravi (80, 75, 90) and Priya (90, 85, 80)

```
--- STUDENT REPORT ---
Ravi Marks: [80.0, 75.0, 90.0] Total: 245.0 Average: 81.67
Priya Marks: [90.0, 85.0, 80.0] Total: 255.0 Average: 85.0
Top student: Priya with total 255.0
Python average: 85.0
Maths average: 80.0
English average: 85.0
```

Copy sample output

Test case 2 — One student

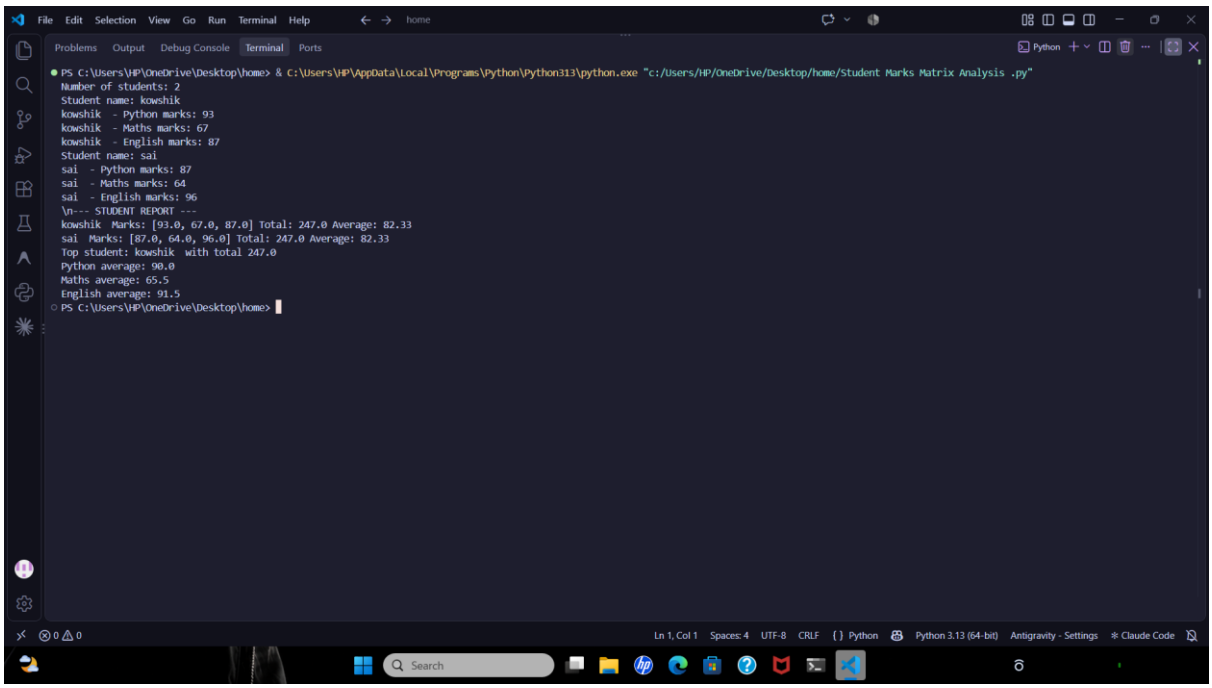
Sample input: Arun with marks 50, 60, and 70

--- STUDENT REPORT ---
Arun Marks: [50.0, 60.0, 70.0] Total: 180.0 Average: 60.0
Top student: Arun with total 180.0
Python average: 50.0
Maths average: 60.0
English average: 70.0

Copy sample output

How to capture actual screenshots:

1. Open the Python file in IDLE or VS Code.
2. Run the program.
3. Enter the sample student names and marks.
4. Capture the console output using Win + Shift + S on Windows.
5. Insert the screenshots into your activity record.



10. Conclusion

The Student Marks Matrix Analysis program was successfully developed using Python. It stores student marks in a two-dimensional list, calculates individual totals and averages, identifies the top student, and finds the average marks for each subject. This program helps us understand lists, nested loops, conditional statements, and matrix operations in Python.